On the translation of Maude programs to modern imperative languages

Rubén Rubio, Beatriz Alcaide García, and Adrián Riesco

Facultad de Informática, Universidad Complutense de Madrid, Madrid, Spain
{rubenrub, balcaide, ariesco}@ucm.es

Abstract. Maude is a high-level specification and programming language based on rewriting logic. Although expressiveness is its primary design goal, Maude achieves a high level of performance. Nevertheless, it is not intended to compete with lower-level imperative languages in terms of raw performance. Moreover, such languages offer greater interoperability with existing software systems. For these reasons, it is worth exploring the mapping of Maude specifications to other programming languages. In this paper, we present a compiler from a subset of Maude to Rust, C++, Python, and Dafny, with particular emphasis on their support for pattern matching and algebraic data types. We evaluate the usefulness of the proposed approach through several simple yet illustrative examples, and assess it by means of differential testing.

1 Introduction

Maude [5] is a high-level specification and programming language based on rewriting logic [20]. Despite its expressivity, its interpreter is also a high-performance rewriting engine. Indeed, in the latest Rewrite Engine Competition [9], Maude comes second just between Haskell and the compiled version of OCaml. In most verification and programming applications of Maude, Maude programs are as efficient as the intended model can be or efficient enough for its purpose.

However, generating semantically-equivalent code in a more conventional programming language may be worth in some situations. One of them is the integration of Maude-based components into other software systems. Maude's built-in support for external objects (sockets, processes, files, etc.) and its Python library [27] are very useful for this purpose, but still need a Maude interpreter process behind the scenes. In embedded or low-resource systems, like robots [18], generating a lightweight standalone program might be required. In this direction, an important reason for compilation is performance. However, compiling even subsets of such a general language like Maude is notably hard and does not guarantee a better performance [7,9]. Compilation has been widely studied and put in practice in the context of declarative programming languages, where different analyses and optimization techniques have been studied. Some intermediate languages, like Prolog's Warren Abstract Machine, have been developed as a compilation target or as a step towards C or machine code. Other rewriting-based languages include compilers like ASF+SDF [4] and ELAN [21].

There have also been abandoned efforts to build a compiler for Maude itself, which are still available in the Maude codebase.

In the verification field, a potentially interesting use case for compilation is statistical model checking [1], where a model with random behavior is executed many times to estimate quantitative values by the Monte Carlo method. In this context, the absence of unquantified nondeterminism (required for the soundness of the statistical analysis) simplifies compilation efforts, which are also rewarded by reusing the same generated code in many executions. Moreover, simulations in statistical model checking are easy to distribute and parallelize, and lightweight specialized code may facilitate exploiting the resources of the host machines.

In this paper, we present an experimental compiler for a subset of Maude specifications to some imperative programming languages, namely Rust, C++, Python, and Dafny. All of these languages but C++ have some support for algebraic datatypes and pattern matching we want to leverage on. While we could have targeted a lower-level language, like LLVM, as compilation target, we wanted to use as much high-level constructs and data structures as possible to simplify the translation and approach the generated code to the original one. Our translation is implemented in two steps: first, an abstract syntax tree for the Maude code is obtained as intermediate representation. Then the tree is used to generate the imperative code. Even though the translation should preserve the semantics by construction, we use differential testing [19] to assess the equivalence of the generated code. For this purpose, we have updated the MUnit framework [26] to work in Maude 3. Some performance evaluations have been carried out with promising results.

The rest of the paper is organized as follows: Section 3 introduces some preliminaries, Section 4 describes the translation from Maude programs to other languages, and Section 5 discusses some case studies. Finally, Section 6 concludes and outlines some lines of future work. The source code of the compiler, examples, and tests are available at `github.com/fadoss/maudec`.

2 Related work

In the early years of rewriting logic, compilation of patterns and rewrite programs attracted much attention. Several abstract machines were proposed like the Reduce ELAN Machine [21], the Rewrite Rule Machine [16], and CafeOBJ's Term Rewriting Abstract Machine [24]. The ELAN language [13] counted with a compiler that produces C code and supports matching modulo theories like AC, rule rewriting, and rewrite strategies. ASF+SDF [4] was also endowed with a compiler. Compilation from a subset of Maude called Simple Maude to SIMD or MIMD machines is discussed in [15]. Maude is an interpreted language, but owes much of its performance to the careful semicompilation of its pattern matching [8]. In this paper, we do not aim to translate Maude to an abstract machine or to a low-level programming language, but to leverage on high-level programming constructs of modern imperative languages.

Soufflé [12] implements a transformation from Datalog [11], a logic programming language used for program analysis in deductive databases, into C++. The main motivation for their translation is improving the performance of the analyses while preserving a declarative interface from the user's perspective. This idea is fully aligned with the approach presented in this paper, which seeks to support rapid system specification alongside efficient execution in widely adopted programming languages.

Regarding the translation to Dafny, we have presented a translation of Maude specifications into the theorem prover Lean [23] in [30]. Dafny and Lean occupy complementary points in the verification space: Dafny emphasizes automation and scalability, while Lean emphasizes expressiveness and proof control. By supporting both, our approach allows users to balance verification effort and assurance, and to apply the most appropriate reasoning technique to each class of properties. Actually, the Maude approach is also complementary to both of them, supporting, for example, model checking and symbolic reachability analysis.

Finally, the Heterogeneous Tool Set (Hets) [22] proposes an approach based on translations, but with a very different nature and purpose. Hets focuses on translations between tools for formal analysis (including Maude), whereas our methodology targets translations between verification-oriented and high-performance languages. Moreover, since the logics supported by Hets are formalized as institutions, translations are formally defined as comorphisms. In contrast, our translations are not formally defined and instead rely on operational techniques such as differential testing. This lack of formalization stems from the nature of the target languages, which do not have a well-established mathematical foundation.

3 Preliminaries

We present here the basic notions of the languages involved in the translation.

3.1 Maude

Maude [6] is a high-performance logical framework based on rewriting logic [20], a logic of change where state transitions are modeled by means of rewrite rules. Rewriting logic is parameterized by an equational logic; in the case of Maude, this logic is membership equational logic [3] which, in addition to equations, allows one to state membership axioms characterizing the elements of a sort.

Maude functional modules [6, Chap. 4], introduced with syntax `fmod` ... `endfm`, are executable membership equational specifications that allow the definition of sorts (by means of keyword `sort`(`s`)); subsort relations between sorts (`subsort`); operators (`op`) for building values of these sorts, giving the sorts of their arguments and result, and which may have attributes such as being associative (`assoc`) or commutative (`comm`), for example; memberships (`mb`) asserting that a term has a sort; and equations (`eq`) identifying terms. Both memberships and equations can be conditional (`cmb` and `ceq`), where conditions have three

variants: (i) equational conditions (t `=` t'), (ii) membership conditions (t `:` s), and (iii) matching conditions (t `:=` t'). In matching conditions the term t possibly includes new variables and holds if t', once reduced by means of equations, matches t. Matching is performed modulo the equational axioms stated in operators, giving Maude specifications a high expressive power. It is worth mentioning that each connected component of sorts by the subsort relation is called a *kind*.

3.2 Some imperative programming languages

C++. C++ [31] is a general-purpose, object-oriented programming language built on top of its precursor C. C++ leverages existing C libraries while gaining additional expressive power and flexibility by means of encapsulation, inheritance, and polymorphism and keeping very good performance. For these reasons, C++ is a widely used language for high-performance applications and large-scale software development. There is no support for pattern matching in the language (although its addition has been proposed), neither for algebraic datatypes, but limited forms can be achieved using types as `std::variant`.

Python. Python [17] is also a general-purpose, object-oriented programming language known for its readability and extensive ecosystem. In recent years, it has gained popularity as the dominant language for artificial intelligence and machine learning, due to its simplicity and the availability of powerful libraries and frameworks. Pattern matching is, in this case, natively supported through structural pattern matching (`match`/`case`).

Rust. Rust [2] is a systems programming language designed to provide memory safety and concurrency without sacrificing performance. It has gained popularity for building reliable and efficient software, particularly in contexts where safety and low-level control are critical. In the case of Rust, pattern matching is a core language feature, deeply integrated via `match`, `if let`, and `while let`.

Dafny. Dafny [25] is a verification-aware programming language with native support for Hoare logic specifications and an SMT-based static program verifier. Pattern matching is supported through `match` statements and expressions.

3.3 Pattern matching support in mainstream languages

Several modern general-purpose imperative programming languages support some form of algebraic data types (ADTs) and pattern matching. Generalizing the traditional `switch` construct, for multiple case distinction in integer values, pattern matching statements and expressions are available in languages like Python and Java since 2021, C# since 2019, Scala, Rust, and Dafny, among others. However, their generality and features differ from one language to the other. Some characteristics that will be relevant when translating Maude statements are:

1. *Nesting* of patterns is supported by all languages in the previous list.

2. *Linearity* proscribes multiple occurrences of the same variable in the pattern. All programming languages mentioned above (and even functional languages like Haskell or OCaml) require their patterns to be linear. Otherwise, the substructures matched by the multiple occurrences of the same variable would need to be deeply checked for equality, which would imply a significant and somehow hidden amount of work. However, a non-linear pattern can be translated to a linear one and some equality checks.
3. *Guards* are conditions that can be attached to each branch of the matching construct to be selected. They cannot be simply replaced by nested conditional statements, because this would avoid the execution of alternative branches of the matching construct. Guards are not supported by Dafny.
4. *Bindings inside guards*, i.e., the possibility of declaring variables in the guards, is only supported by Python (and Maude). However, this only makes a difference with languages like Rust or C++, where variable assignment means potentially non-trivial copy or movement of values.
5. *Disjunction of patterns* allows assigning the same body to different patterns with the same variables, usually with the syntax p_1 `|` $\cdots$ `|` p_n. However, its semantics may differ between languages when combined with a guard. In Rust, matching with the disjunctive pattern succeeds if there is a matching subpattern that satisfies the condition, while in Python, this happens only when the first matching subpattern satisfies the condition. In Dafny, disjunctive patterns cannot bind variables.
6. *Matching on predefined data structures*, like tuples or lists, is supported with limitations by most languages. For instance, Rust supports *slice patterns* `[`p_1`,` p_2`,` r `@ ..,` p_3`]` with some element subpatterns and at most one slice capture. This is similar to sequence patterns in Python p_1`,` p_2`, *`r`,` p_3, where at most one starred element (representing a slice) is allowed. These restrictions are reasonable, since having multiple slice components will imply a search and increase the complexity of pattern matching. Dafny does not support list patterns.

Pattern matching is usually applied on instances of algebraic datatypes. Out of the languages mentioned before, only Scala, Rust, and Dafny support algebraic datatypes, often called *sum types* or *enumerations* with fields. For example, in Rust, a type that either holds a value of type `T` or nothing is declared as:

```
enum Option<T> { None, Some(T), }
```

In other languages, ADTs can be implemented as tagged unions using enumeration constants, classes, records, or tuples. In C and C++, they are often implemented with `union` and a tag of an `enum` type, but C++17 introduced them in the standard library with `std::variant`. Another key difference between languages is whether the responsibility of memory management is on the user (i.e., garbage collection, explicit allocation and deallocation of resources, etc.). This makes implementing recursive data structures easy in Python, Java, or Dafny, but not in C++ or Rust. Indeed, Rust sum types cannot be recursive unless explicit (smart) pointers are used.

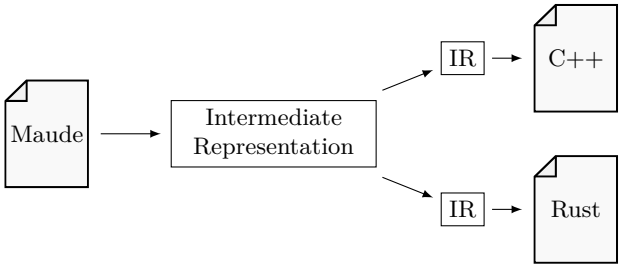

Fig. 1. Architecture of the translation.

In the implemented tool [29], whose principles are described in Section 4, Rust, Python, C++, and Dafny are supported as target languages. Moreover, for simplicity, we will use Rust as the main language for the examples in this paper.

3.4 Testing

We briefly describe the unit testing frameworks available for the languages involved in the translation. MUnit [26] is a framework for Maude specifications that we have reimplemented to work in Maude 3.[1] Although it provides assertions specific to Maude, here we are interested in `assertTrue(`t`)` (the given term t is reduced to `true`), `assertFalse(`t`)` (the term t is reduced to `false`), `assertEquals(`t`, `t'`)` (the normal forms of t and t' are the same) and `assertLeqSort(`t`, `s`)` (the normal form of t has sort s).

MUnit is inspired by Python's `unittest`,[2] so they are very similar. It also provides `assertTrue`, `assertFalse`, and `assertEqual`, with the expected meaning. In turn, Rust's unit test infrastructure [14, § 11] provides the assertions `assert!(`t`)` and `assert_eq!(`t`, `t'`)`, along with syntax to define functions as tests. Regarding Dafny, it also includes built-in support for writing test as functions annotated with the `:test` attribute and also to generate them based on the specification. Finally, for C++, we implement the tests ourselves.

4 Code generation from Maude programs

In this section, we explain the proposed code generation from a subset of Maude programs, which goes through an intermediate internal representation. Only a subset of Maude is implemented in the tool [28], so we will indicate what is supported and how the transformation can be extended beyond the implementation. Given a Maude module, the translation consists of several steps, as illustrated in Figure 1:

[1] The new version is available at https://github.com/ariesco/MUnit.
[2] https://docs.python.org/es/3/library/unittest.html

1. The Maude module is analyzed to index module elements (kinds, sorts, operators, etc.), identify constructors, locate built-in types, characterize sorts, calculate the kind-containment relation, etc. This information is a valuable base for the next steps.
2. An abstract syntax tree is built in an intermediate representation (explained in Section 4.1) from the Maude module. This process is described in Section 4.2.
3. Some transformations on the intermediate representation are applied to simplify it or adapt it to the desired target language. For instance, `match` statements can be removed for languages like C++. We explain it in Section 4.3.
4. Code in the final language is generated from the intermediate representation, as explained in Section 4.4.

The module information is obtained through the Maude Python library [27] and it is later processed in Python. One relevant relation that is calculated at this stage is the mutual dependency or containment between kinds. A kind k directly depends on another k' if there is an operator with range in k and a sort of kind k' in its domain. The non-trivial strongly-connected components of this relation are the sets of recursive or mutually-recursive kinds.

4.1 Intermediate representation

Using an intermediate representation is a common practice in compilers and language processors, not only when multiple inputs or outputs are planned, but also to apply transformations or optimizations in a more convenient setting. Our intermediate representation is designed to take advantage of the support for algebraic datatypes and pattern matching in the potential target languages, while providing alternatives to circumvent the lack of pattern matching after an internal transformation.

A program in this intermediate representation is a collection of sum type declarations, function declarations, and their bodies, internally represented as objects in some hierarchies `Node` of syntactic nodes, `Pattern` of patterns, and `AstType` of types (see Figure 2). The definitions of these elements are the expected ones: function declarations are top-level nodes with a name, a sequence of argument names and types, and a range type; expressions of type `Expr` are a particular category of nodes that yield a value (and have a type) and comprises constants, variables, function, operator, and datatype constructor calls, and conditional expressions, among others. Other syntactic nodes that are not expressions are variable declarations, conditional blocks, and return statements. Types can be simple (`NamedType`) or compound (`PointerType`, `SequenceType`, etc.), where named types of name `bool`, `int`, `nat`, `float`, and `string` are intended to match the corresponding built-in types in Maude and the target language. User-defined algebraic datatypes are also named types, but they are declared in a `SumTypeDecl` with a name and sequence of `Variant`s, which in turn are given by a name and a sequence of argument types. For instance, an algebraic datatype with two variants that contains either an integer or none is written as

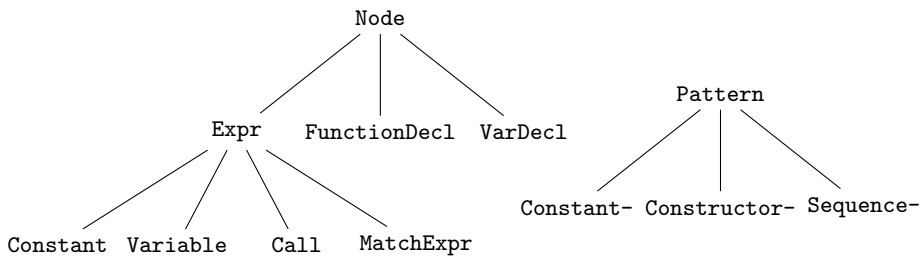

Fig. 2. Summary of the main elements of the intermediate language.

```
SumTypeDecl('IntOption', [Variant('None')],
      Variant('Some', [NamedType('int', builtin=True)])])
```

Sum types are destructured by pattern matching with the `MatchExpr`, i.e., a pattern-matching expression like those we have discussed in Section 3.3. It is given by an input expression and a sequence of branches, where each branch consists of a pattern of type `Pattern` and an optional Boolean guard of type `Expr`. There are several subclasses of patterns: `VariablePattern`, `ConstantPattern`, `ConstructorPattern` for patterns on algebraic datatypes, `OrPattern` for disjunctions of patterns, `TuplePattern`, and `SequencePattern` for patterns on lists. Most of these patterns exist in the target languages, although with some limitations, as pointed out in Section 3.3.

With these rudiments, we build the translations from Maude and to the target languages as explained in the following sections.

4.2 From Maude to the intermediate representation

The translation from a Maude module to the intermediate representation and to the target language aims to be as direct as possible, although this is limited by the lower generality of this representation and the target languages. The unit of translation is a flattened Maude module and the modular structure is not preserved. In summary, kinds are translated to types, constructor operators become data constructors for those types, equationally-defined operators are translated to functions whose body is a `match` expression, and terms are recursively translated to expressions on built-in constants, variables, operators, function, and data constructors calls. Let us describe the details element by element.

For each kind k, we declare a sum type $T(k) =$ `NamedType(`k`)` with a variant of domain $T^*([s_1]) \cdots T^*([s_n])$ for each constructor `op` f `:` $s_1 \cdots s_n$ `->` s `[ctor]` in the kind. We denote by $[s]$ the kind of the sort s, and $T^*(k') =$ `PointerType(`$T(k')$`)` if k and k' are mutually-recursive kinds or $T^*(k') = T(k')$ otherwise. For example, the Maude code

```
op mt : -> Stack [ctor] .
op push : Int Stack -> Stack [ctor] .
op empty : Stack -> Bool .
```

gives the following sum type

```
SumTypeDecl('Stack', [Variant('mt', []),
    Variant('push', [NamedType('nat', builtin=True),
                     PointerType(NamedType('Stack'))])])
```

Built-in types and operators with structural axioms are handled differently:

- The constructors of the built-in sorts `Bool`, `Nat`, `Int`, `Float`, and `String` are recognized by their attributes (since names may have changed by a renaming) to translate their instances to Boolean, integer, and string constants in the intermediate representation. If the built-in type constructors are the only one in their kinds, no sum type is defined and the type is used unwrapped. Otherwise, the built-in type is one of the variants of the sum type.
- Commutative constructors are represented as in the general case, but a function is defined to normalize them by sorting their arguments. When the operator is applied on non-ground arguments, this function is called instead of the sum type constructor.
- Iterative symbols `op` f `:` s `->` s `[iter]` are translated to a variant with signature $T^*([s])$ `NamedType('nat', builtin=True)`, where the second argument is the exponent. A normalizing function is similarly defined.
- Associative operators (with or without unity) are translated to sequences or lists. When the identity element is a nullary constant (in Maude, identity elements can be any term), it is identified with the empty sequence. If the associative constructor cannot be nested in other constructors of the kind, $T(k)$ is defined without that constructor and `SequenceType(`$T(k)$`)` is used in place of sorts containing the operator. Otherwise, a variant of that sequence type is added to the sum type. Associative operators are not fully supported in the implementation, specially when matching with unsupported patterns in the target languages.
- Associative and commutative (AC) operators can be represented as multisets in the same way associative operators are translated to sequences. However, unlike for sequences, there is no support for matching inside multisets in the considered imperative languages, so it must be implemented explicitly. Notice that efficient matching modulo AC is complicated [13,8], but some usual patterns can be more easily, yet not trivially, handled with a search. Our implementation does not support these operators yet.

Examples with commutative and associative operators can be found in Section 5.

Even though data constructors are defined at the kind level, sorts need to be taken into account for matching and sort-membership tests. Moreover, they are useful for simplification of the generated code. Explicit $t :: s$, $t : s$, and implicit sort-membership tests are usually translated to calls to `hasSort_`s predicates. These predicates are defined by a match expression with the constructor declarations and membership axioms in the sort, and potentially using the predicates for other sorts. For built-in types, specialized inline tests are generated, like $x \geq 0$ for `Nat` (if not the only maximal sort) or $x > 0$ for `NzNat`. We do not attach

sort information to the objects, which will make sort testing more efficient, so programs relying on complex sort calculations will not work efficiently.

Each operator with equations is translated to a function, whose domain and result type are obtained by translating the corresponding Maude kinds. It is defined by a single `match` expression on a tuple with the function arguments, and with a case for each equation that has that symbol on the top of its left-hand side. The case patterns are obtained by translating the left-hand side of the equations, and the case body is the translation of the right-hand-side term. Conditions (if any) are inserted into the case guard using the equality operator of the datatype and the aforementioned sort-membership functions. Matching conditions introduce variable declarations in the case body for the free variables in their left-hand sides. For example, the `empty` function declared above can be defined in Maude with

```
eq empty(mt) = true .
eq empty(push(N:Int, S:Stack)) = false .
```

This is translated to a function in the internal representation equivalent to the more easily readable Rust code

```
fn empty(s: &Stack) -> bool {
  match s { Stack::Mt => true,
            Stack::Push(_, _) => false, }
}
```

In case a subterm occurs multiple times in an equation, we use an auxiliary variable declaration to avoid recomputing the subterm multiple times (in place of the hash consing of Maude). Built-in operators (called *special* in the Maude terminology) are translated to calls that will be eventually translated to built-in operators or library function calls in the target language. This includes Boolean and arithmetic operators, trigonometric functions, etc. Structural axioms are taken into account when constructing patterns. For example, the left-hand side f`(`p_1`, `p_2`)` where f is a commutative operator is translated to the disjunctive pattern `(`p_1`, `p_2`) | (`p_2`, `p_1`)`.

Support for rules in the current version of the translation is limited to deterministic rewriting, although it can be extended to deal with nondeterminism. We introduce a function for each kind that applies any possible rule on top of the given term. The definition of this function is similar to those of equationally-defined operators. For example, the following two rules describe the Collatz sequence, whose termination is only a conjecture as of today.

```
crl N => N quo 2   if N rem 2 = 0 .
crl N => 3 * N + 1 if N rem 2 = 1 .
```

They are translated to an intermediate representation that will generate the following Rust code

```
fn rlapp_Nat(term: i32) -> Option<i32> {
  match term {
    n if n % 2 == 0 => Some(n / 2),
```

```
    n if n % 2 == 1 => Some(3 * n + 1),
    _ => None,
  }
}
```

Notice that it returns an `Option<i32>` instead of the raw value, so that we are able to recognize when rules cannot be applied. Then, we also define some functions, called `arlapp_`k, to rewrite the innermost leftmost regex within the given term. They delegate on `rlapp_`k and other `arlapp_`k' to rewrite on top and inside subterms, respectively. Rewriting conditions t `=>` t' are currently not supported, but they can be implemented by a nested search. For introducing nondeterminism, the rule-application functions can be defined as coroutines to incrementally provide solutions. In that case, a single `match` expression would exclude rewrites, since only the first matching case is activated, so patterns should be checked one by one. Rewrite strategies can then be translated to functions that invoke these rule functions.

4.3 Transformations within the intermediate representation

The intermediate representation generated from the Maude code can be further processed, simplified, or optimized. For instance, match expressions on tuples (which are used for the argument of a function) are simplified by removing those variables in which there is no case distinction. In addition to the generic transformations, the syntax tree may be also adapted to a particular target language or family of them. Some of these transformations are:

- The *dematch* transformation completely removes the `match` expressions and replaces them by equivalent code with conditionals that explictly checks in which variant the object is and extracts the information from it, using the `VariantCheck` and `VariantAccess` expressions of the intermediate language. This is useful for C++, where pattern matching is not available.
- Rust does not support patterns that traverse pointers (although there are proposed features like `box_patterns` and `defer_patterns` in unstable Rust to overcome this problem). Hence, we replace the pointer subpattern by a variable and translate it as a condition appended to the guard and some variable declarations.
- Since the semantics of pattern disjunction with guards in Python differ from our intended meaning, a transformation for eliminating disjunctive patterns is applied. It translates a guarded branch with a pattern disjunction into several branches, after resolving all disjunctions that deeply appear in the pattern. This implies copying the guard and body several times.

An example of the *dematch* transformation is shown in Section 5.

4.4 From the intermediate representation to the target languages

Once the intermediate representation is available and conveniently transformed, code generation in the target language is almost straightforward. Some adaptations are required for languages that do not support name overloading (Python

and Rust), in which the declaration order is relevant (C++, a topological order of function dependencies is calculated), etc. However, the most critical concern at this point is memory management, which is explicit in C++ and Rust. For convenience, we take arguments by constant reference (in C++) or borrow them (in Rust), always return copies of objects, and use smart pointers in recursive data types. When reducing with equations or rewriting with rules, modifying the term in place would be more efficient and should be considered as future work. The general principle of the translation is to generate code as conventional as possible in the target language, so implementing a significant runtime for term manipulation or *hash consing* [10] is a non-goal.

4.5 Testing the translation

Differential testing [19] is a testing technique in which the same input is executed on multiple independent implementations of a system, and their outputs are compared to detect inconsistencies. We have written several Maude specifications along with some MUnit tests. On the one hand, these tests are executed by the MUnit tool on the Maude specification. On the other hand, our code generator takes them as input, recognizes `assertTrue`, `-False`, `-Equal`, and `-LeqSort` tests, and generates tests in the target language using the frameworks mentioned in Section 3.4. A simple script `tester.py` is provided to generate the code from the Maude specification, build it, and run the tests in the target languages.

5 Examples

This section illustrates the translation of Section 4 with several examples. We have run small experiments for some of them to compare the performance between the different target languages and Maude. The experiments ran under Linux in an Intel Xeon machine, with the official build of Maude 3.5.1, Rust 1.92.0 (without or with the `-O` optimization flag), GCC 13.3.0 as the C++ compiler (without or with the `-O3` flag), CPython 3.12.3, and PyPy 7.3.20.

5.1 Numeric functions

The `Bool`, `Nat`, `Int`, and `Float` types in Maude are translated to the corresponding native types of the target language. This can be observed with the following Fibonacci function:

```
op fib : Nat -> Nat .
eq fib(0) = 0 .  eq fib(1) = 1 .
eq fib(s s N) = fib(s N) + fib(N) .  var N : Nat .
```

Then, the code generator can be run with the following command

```
$ maudec tests/nat-funs.maude --target rust
```

to produce the following Rust code

	Maude	Rust	-O	C++	-O3	CPython	PyPy
fib	3 min 57 s	3.4 s	893 ms	2.65 s	560 ms	1 min 9.9 s	10.85 s
eeuclid	9.58 s	121.6 ms	60.4 ms	269.5 ms	57.9 ms	1.22 s	247 ms
minimum	26.68 s	226.4 ms	2.1 ms	86.3 ms	1.4 ms	108.8 ms	107.1 ms
palindr	50 ms	14.2 ms	3 ms	30.2 ms	7.5 ms	52.34 s	61.16 s

Table 1. Performance comparison of various examples in the different languages.

```
fn fib(a1: i32) -> i32 {
  match a1 {
    0 => 0, 1 => 1,
    n if n >= 2 => fib((n - 2) + 1) + fib(n - 2),
    _ => panic!("case not covered"),
  }
}
```

Observe that the guard `n >= 2` and the fallback case could be removed if we found out that they are redundant (i.e., the equations are complete). However, we cannot drop the fallback case without removing the guard, because the Rust compiler will complain that the cases are not exhaustive. Moreover, the guard is the result of translating the pattern `s s N` of the third equation, while the occurrences of `N` in the right-hand side have been translated to `n - 2`.

We have measured the time for computing all Fibonacci numbers from 1 to 40. Table 1 shows that this takes almost 4 minutes in Maude, 839 ms in an optimized build of the generated Rust code, 560 ms with an optimized C++ build, and almost 11 seconds for the Python program run with PyPy. We should take into account that the C++ and Rust translation uses machine integers, while Maude and Python use big integers, which are slower. This example is artificial, since memoization can drastically reduce the computation effort. In effect, with the `memo` flag in the `fib` operator [5, §3.4.8], Maude runs the experiment in 1 ms.

A more interesting and complex example is the following specification of the extended Euclidean algorithm.

```
sort EuclidResult .
op euc : Int Int Int -> EuclidResult [ctor] . *** result
op eeuclid : Int Int -> EuclidResult .
op eeuclid : Int Int Int Int Int Int -> EuclidResult .
vars N M U1 V1 U2 V2 : Int .
eq eeuclid(M, N) = eeuclid(M, 1, 0, N, 0, 1) .
eq eeuclid(M, U1, V1, 0, U2, V2) = euc(M, U1, V1) .
eq eeuclid(M, U1, V1, N, U2, V2) =
     eeuclid(N, U2, V2, M - (M quo N) * N,
             U1 - (M quo N) * U2, V1 - (M quo N) * V2) .
```

The result of `eeuclid(`m`, `n`)` is a term `euc(`d`, `x`, `y`)` with the greatest common divisor d of m and n, and the Bézout coefficients x and y. The tool yields the following Rust code:

```
#[derive(Clone, PartialEq, PartialOrd)]
enum EuclidResult { Euc(i32, i32, i32), Err, }

fn eeuclid(m: i32, u1: i32, v1: i32,
           n: i32, u2: i32, v2: i32) -> EuclidResult {
  match n {
    0 => EuclidResult::Euc(m, u1, v1),
    n => { let aux1 : i32 = m / n;  // repeated subterm
      eeuclid(n, u2, v2, m - n * aux1,
              u1 - u2 * aux1, v1 - v2 * aux1)
    },
}}
fn eeuclid_1(m: i32, n: i32) -> EuclidResult {
  eeuclid(m, 1, 0, n, 0, 1)
}
```

where `eeuclid_1` and `eeuclid` are the two overloads of `eeuclid` in the Maude program, and the `euc` constructor is now the `Euc` variant of the sum type. `EuclidResult` also contains an *error term* `Err` for partial functions. Inserting this is avoided for kinds that are considered error-free by Maude.

In this case, we have measured the time to compute `euclid` on every input in $\{1, \ldots, 1000\}^2$, and the results appear in the second row of Table 1. Maude takes around 9.5 seconds, while the optimized Rust and C++ builds take approximately 60 ms. It seems that numerical functions can be significantly accelerated with the translation.

5.2 Commutative operator

The minimum of two numbers can be defined in Maude with this simple conditional equation on a commutative operator:

```
op minimum : Nat Nat -> Nat [comm] .
ceq minimum(M, N) = M if M <= N .  vars M N : Nat .
```

This is translated to the following Rust code

```
fn minimum(n: i32, m: i32) -> i32 {
  match (n, m) { (n, m) | (m, n) if m <= n => m, }
}
```

where commutativity is handled by the disjunction of patterns, so that the two possible orderings of the arguments are tried. We can also produce C++ code:

```
int minimum(int N, int M) {
  if (M <= N) { return M; } if (N <= M) { return N; }
}
```

The body of the function has been *dematched* in the internal representation to the two conditional statements. Both translations are artificial, but correct and relatively efficient, as shown in the third row of Table 1 for its evaluation on $\{1, \ldots, 5000\}^2$. Given the high number of inputs, the measurement for Maude

may be affected by the recursive function that iterates over them to call the `minimum`, which is implemented by two loops in the other languages.

Since this operator is not a constructor, we do not generate both a variant constructor and a normalization function, but only the function itself. See the `unordered-pair` test in [29] for an example of a commutative constructor.

5.3 Stacks

In Section 4.2, we introduced stacks of integer values and defined an `empty` predicate on them. Such data structures are usually specified as parameterized modules, like `fmod STACK{X :: TRIV}`. However, we have not considered translating parameterized types to generic types in the target language, so we only work with instances of these types. The definition of the datatype will be

```
#[derive(Clone, PartialEq, PartialOrd)]
enum Stack { Mt, Push(i32, Box<Stack>), Err, }
```

where `Mt` represents the empty list, `Push` appends an element to the stack, and `Err` is the error term. Observe that the recursive argument of `Push` is wrapped into a `Box`, which is a unique pointer to a heap allocated object. Hence, the implementation has to deal with this pointer explicitly.

In addition to the `empty` predicate defined in Section 4.2, we can define a partial function `pop` to remove the top element of the stack.

```
op pop : Stack ~> Stack .
eq pop(push(N, S)) = S . var N : Int . var S : Stack .
```

This is translated to the Rust code

```
fn pop(a1: &Stack) -> Stack {
  match a1 {
    NatStack::Push(_, paux) => (**paux).clone(),
    _ => Err,
  }
}
```

where a copy of the nested stack is returned (the copy can be avoided with a more sophisticated memory management). Moreover, there is a fallback case that returns the term `Err` when the partial function is invoked out of its domain.

5.4 Palindromes

Consider the following module with a function that checks whether a word on some vowel symbols is a palindrome:

```
fmod PALINDROME is
  sorts Symbol List . subsort Symbol < List .
  ops a e i o u : -> Symbol [ctor] .
  op nil : -> List [ctor] .
  op __ : List List -> List [ctor assoc id: nil] .
```

	Maude	Rust	-O	C++	-O3	CPython	PyPy
fib	7.63 Mb	1.74 Kb	1.74 Kb	73.73 Kb	73.73 Kb	1.23 Mb	107.73 Mb
eeuclid	7.65 Mb	1.74 Kb	1.74 Kb	73.73 Kb	73.73 Kb	1.35 Mb	16.57 Mb
minimum	7.05 Mb	1.74 Kb	1.74 Kb	73.73 Kb	73.73 Kb	1.23 Mb	13.92 Mb
palindr	23 Mb	66.1 Kb	66.1 Kb	198 Kb	198 Kb	7.69 Gb	9.12 Gb

Table 2. Peak memory usage of the same examples and languages in Table 1.

```
  op palindrome : List -> Bool .
  vars S S1 S2 : Symbol . var L : List .
  eq palindrome(nil) = true . eq palindrome(S) = true .
  eq palindrome(S L S) = palindrome(L) .
  eq palindrome(S1 L S2) = false [owise] .
endfm
```

Our tool translates it to the following type declaration and function using the slice patterns of Rust (which are similar to Python's).

```
#[derive(Clone, PartialEq, PartialOrd)]
enum Symbol { A, E, I, O, U, }

fn palindrome(s: &[Symbol]) -> bool {
  match s { [] => true, [_] => true,
    [s1, l @ .., s2] if s1 == s2 => palindrome(l),
    [_, .., _] => false,
  }
}
```

Because the list constructor cannot appear in other constructors of the kind, it has been excluded from the datatype definition and a sequence type in Rust (`&[Symbol]`) is used, as explained in Section 4.2.

In Table 2, we can observe a significant reduction in memory usage when translating to Rust and C++, but a huge rise when translating to Python. Slices in Rust and C++ are views over a memory segment of the original list (essentially a pointer and a length), while they are full copies in Python. Table 1 also shows a much greater execution time for Python.

5.5 Rule application

As explained in Section 4, the translation defines some functions to execute single-step rewrites within the given term for those kinds where a rewrite is possible. Here we illustrate them with a small example, whose original Maude code is the following:

```
mod NESTED-REWRITE is
  sorts Foo Bar .
```

```
  op f : Bar Foo -> Foo [ctor] .
  ops a b c : -> Bar [ctor] .

  rl [ab] : a => b .
  rl [bc] : b => c .
endm
```

In Rust, the compiler generates the following data definitions:

```
#[derive(Clone, PartialEq, PartialOrd, Debug)]
enum Bar { A, B, C, }
#[derive(Clone, PartialEq, PartialOrd, Debug)]
enum Foo { F(Bar, Box<Foo>), }
```

and a function `rlapp_Bar` that applies either `ab` or `bc` (i.e., all rules defined on the kind `Bar`) on top of the given term:

```
fn rlapp_Bar(term: &Bar) -> Option<Bar> {  // on top
  match term {
    Bar::A => Some(Bar::B),
    Bar::B => Some(Bar::C),
    _ => None,
  }
}
```

The return value can be either `Some`(t') for a rewritten term t', or `None` if no rewrite is possible. For rewriting anywhere within a term, two more functions are provided, `arlapp_Bar` and `arlapp_Foo`, one for each kind.

```
fn arlapp_Bar(term: &Bar) -> Option<Bar> {
  rlapp_Bar(term)  // only possible on top
}

fn arlapp_Foo(term: &Foo) -> Option<Foo> {
  match term {
    Foo::F(t0, t1) => {
      if let Some(subt) = arlapp_Bar(t0) {
        return Some(Foo::F(subt, (*t1).clone()));
      }
      if let Some(subt) = arlapp_Foo(&**t1) {
        return Some(Foo::F((*t0).clone(), subt.into()));
      }
    },
  }
  None  // no rewrite possible on top
}
```

The `arlapp_Foo` function decomposes the terms of kind `Foo`, tries to apply the `arlapp` functions recursively on the arguments, and finally returns the recomposed rewritten term if the nested rewrite succeeded. The case of the second argument is more involved, since this argument is recursive and mediated by a pointer (`Box<Foo>`).

6 Conclusions and ongoing work

In this paper we have described a translation from Maude functional modules to different modern imperative languages. The translation starts from a Maude specification and builds an abstract syntax tree, which is used as intermediate language. This tree is later used for generating code in C++, Rust, Python, and Dafny, which provide features that facilitate a natural translation, despite the significantly higher expressiveness of Maude specifications. We have also implemented a translation for test cases, hence supporting differential testing and providing a higher level of confidence. Our experiments with simple examples show that the obtained implementations are significantly faster than the original Maude code. Moreover, the translation to Dafny expands the spectrum of verification techniques.

The current implementation establishes a solid foundation for extensions. We are currently extending the subset of Maude specifications supported by the translation. It is especially interesting to analyze how to integrate ACU operators and rewrite rules. Finally, we aim to assess our approach using a larger set of benchmarks. These benchmarks should include a wide range of applications, including larger specifications than the ones shown in this paper, the new features under development, and also specifications that require proving properties with Dafny, illustrating how Maude verification techniques are complemented.

We also consider studying the usage of artificial intelligence (AI) tools in different directions. A promising application of AI lies in the automated generation of test cases, where it can efficiently explore a wide range of input scenarios. We will take advantage of this fact for obtaining larger test suites. Moreover, AI can be asked to improve the efficiency of the generated code, as it works better with mainstream programming languages. In this case, we would analyze how the code evolves and take advantage of tests for assessing equivalence.

Acknowledgments. We would like to thank Narciso Martí-Oliet for his comments on this manuscript. This work is partially supported by MCIU/AEI/10.13039/50110-0011033/FEDER,UE through project ProCode 10 (PID2023-149943OB-I00) and by Comunidad de Madrid/UCM through project COMFIA (PR17/24-31913).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the contents of this article.

References

1. Agha, G., Palmskog, K.: A survey of statistical model checking. ACM Trans. Model. Comput. Simul. **28**(1), 6:1–6:39 (2018). https://doi.org/10.1145/3158668
2. Blandy, J., Orendorff, J., Tindall, L.F.S.: Programming Rust: Fast, Safe Systems Development. O'Reilly, 2nd edn. (2021)
3. Bouhoula, A., Jouannaud, J., Meseguer, J.: Specification and proof in membership equational logic. Theor. Comput. Sci. **236**(1-2), 35–132 (2000). https://doi.org/10.1016/S0304-3975(99)00206-6

4. van den Brand, M., Heering, J., Klint, P., Olivier, P.A.: Compiling language definitions: the ASF+SDF compiler. ACM Trans. Program. Lang. Syst. **24**(4), 334–368 (2002). https://doi.org/10.1145/567097.567099
5. Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.L.: Maude Manual v3.5.1 (July 2025), https://maude.cs.illinois.edu/manual
6. Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Talcott, C.L. (eds.): All About Maude - A High-Performance Logical Framework, LNCS, vol. 4350. Springer (2007). https://doi.org/10.1007/978-3-540-71999-1
7. Denker, G., Talcott, C.L., Rosu, G., van den Brand, M., Eker, S., Serbanuta, T.: Rewriting logic systems. In: WRLA 2006. ENTCS, vol. 176(4), pp. 233–247. Elsevier (2006). https://doi.org/10.1016/j.entcs.2007.06.018
8. Eker, S.: Fast matching in combinations of regular equational theories. In: Meseguer, J. (ed.) RWLW 1996. ENTCS, vol. 4, pp. 90–109. Elsevier (1996). https://doi.org/10.1016/S1571-0661(04)00035-0
9. Garavel, H., Tabikh, M., Arrada, I.: Benchmarking implementations of term rewriting and pattern matching in algebraic, functional, and object-oriented languages - the 4th rewrite engines competition. In: Rusu, V. (ed.) WRLA 2018. LNCS, vol. 11152, pp. 1–25. Springer (2018). https://doi.org/10.1007/978-3-319-99840-4_1
10. Goto, E.: Monocopy and associative algorithms in extended Lisp. Tech. rep., University of Tokyo (1974)
11. Hafner, C.D., Godden, K.: Portability of syntax and semantics in DATALOG. ACM Transactions on Information Systems **3**(2), 141–164 (Apr 1985). https://doi.org/10.1145/3914.3982
12. Jordan, H., Scholz, B., Subotic, P.: Soufflé: On synthesis of program analyzers. In: Chaudhuri, S., Farzan, A. (eds.) CAV 2016, Part II. LNCS, vol. 9780, pp. 422–430. Springer (2016). https://doi.org/10.1007/978-3-319-41540-6_23
13. Kirchner, H., Moreau, P.: Promoting rewriting to a programming language: a compiler for non-deterministic rewrite programs in associative-commutative theories. J. Funct. Program. **11**(2), 207–251 (2001). https://doi.org/10.1017/S0956796800003907
14. Klabnik, S., Nichols, C., Krycho, C., et al.: The Rust Programming Language. No Starch Press (2026)
15. Lincoln, P., Martí-Oliet, N., Meseguer, J., Ricciulli, L.: Compiling rewriting onto SIMD and MIMD/SIMD machines. In: Halatsis, C., Maritsas, D.G., Philokyprou, G., Theodoridis, S. (eds.) PARLE'94. LNCS, vol. 817, pp. 37–48. Springer (1994). https://doi.org/10.1007/3-540-58184-7_88
16. Lincoln, P., Meseguer, J., Ricciulli, L.: The rewrite rule machine node architecture and its performance. In: Buchberger, B., Volkert, J. (eds.) CONPAR 94 - VAPP VI. LNCS, vol. 854, pp. 509–520. Springer (1994). https://doi.org/10.1007/3-540-58430-7_45
17. Lutz, M.: Programming Python. O'Reilly, 4th edn. (2011)
18. Martin-Martin, E., Montenegro, M., Riesco, A., Rodríguez-Hortalá, J., Rubio, R.: Verification of the ROS NavFn planner using executable specification languages. J. Log. Algebraic Methods Program. **132**, 100860 (2023). https://doi.org/10.1016/j.jlamp.2023.100860
19. McKeeman, W.M.: Differential testing for software. Digit. Tech. J. **10**(1), 100–107 (1998)
20. Meseguer, J.: Twenty years of rewriting logic. J. Log. Algebr. Program. **81**(7-8), 721–781 (2012). https://doi.org/10.1016/j.jlap.2012.06.003

21. Moreau, P.: REM (reduce elan machine): Core of the new ELAN compiler. In: Bachmair, L. (ed.) RTA 2000. LNCS, vol. 1833, pp. 265–269. Springer (2000). https://doi.org/10.1007/10721975_19
22. Mossakowski, T., Maeder, C., Lüttich, K.: The heterogeneous tool set, Hets. In: Grumberg, O., Huth, M. (eds.) TACAS 2007. LNCS, vol. 4424, pp. 519–522. Springer (2007). https://doi.org/10.1007/978-3-540-71209-1_40
23. de Moura, L.M., Kong, S., Avigad, J., van Doorn, F., von Raumer, J.: The Lean theorem prover. In: CADE 2015. LNCS, vol. 9195, pp. 378–388. Springer (2015). https://doi.org/10.1007/978-3-319-21401-6_26
24. Ogata, K., Ohhara, K., Futatsugi, K.: TRAM: an abstract machine for order-sorted conditioned term rewriting systems. In: Comon, H. (ed.) RTA-97. LNCS, vol. 1232, pp. 335–338. Springer (1997). https://doi.org/10.1007/3-540-62950-5_84
25. Project, D.: Dafny, https://dafny.org/
26. Riesco, A.: MUnit: A unit framework for Maude. In: Rusu, V. (ed.) WRLA 2018. LNCS, vol. 11152, pp. 45–58. Springer (2018). https://doi.org/10.1007/978-3-319-99840-4_3
27. Rubio, R.: Maude as a library: an efficient all-purpose programming interface. In: Bae, K. (ed.) WRLA 2022. LNCS, vol. 13252, p. 14. Springer (2022). https://doi.org/10.1007/978-3-031-12441-9_14
28. Rubio, R., García, B.A., Riesco, A.: An experimental Maude compiler to modern imperative languages (2026), https://github.com/fadoss/maudec
29. Rubio, R., Riesco, A.: Maude to Lean translator (2024), https://github.com/fadoss/maude2lean
30. Rubio, R., Riesco, A.: Maude2Lean: Theorem proving for Maude specifications using Lean. J. Log. Algebraic Methods Program. **142**, 101005 (2025). https://doi.org/10.1016/j.jlamp.2024.101005
31. Stroustrup, B.: The C++ Programming Language. Addison Wesley, 4th edn. (2014)